DA4: Best Practices for Project Repository

Jack Donnelley

Reminder

- Completion of this DA4 is due: October 16th, 2026 **(today)**.
- Topic and data for final project is due: October 16th, 2026 **(today)**.

Introduction

As we discussed in DA3, GitHub is a platform that facilitates collaboration on coding and data science projects. In fact, for your final project, you are expected to create a GitHub repository that contains all your reproducible code and proper documentation that explains the structure and steps of the project. Following good practices from the beginning can be helpful since they:

- Help keep your code organized and clear. It makes it easy to find where each step in the analysis is performed.
- It makes it easier for others to read and reproduce your code.
- It is easier to work collaboratively, since it helps divide and organize tasks.

In the present DA4, we explore some good practices that can help with well-maintaining your code and repository.

Step 0: Naming

Naming is vital to keeping a repository organized. One useful approach is to number files so that they appear in a logical order. Complete the following step.

0a) Rename this file to `11_nbaAnalysis_dataProcessing.Rmd`. You will see the usefulness of this numbering convention as we move forward.

Step 1: Creating a `00_requirements.R` File

When running a data analysis in R, it is often useful to know what packages you need for the analysis. One easy way to do this is by creating an **R package setup script** that installs and loads all packages required for your analysis. With this in mind, complete the following steps.

1a) In the folder that you created to run DA4, create a plain R-code file and name it `00_requirements.R`

1b) Use the example code shared and expand it to install and include the packages `tidyverse`, `readxl` and `magrittr`.

```
if (!require(tidyverse)) {       ## If package is not installed
  install.packages("tidyverse") ## Install
  library(tidyverse)            ## Load package to environment.
}
```

```
## Loading required package: tidyverse
```

```
## -- Attaching core tidyverse packages ------------------------ tidyverse 2.0.0 --
## v dplyr     1.2.1     v readr     2.2.0
## v forcats   1.0.1     v stringr   1.6.0
## v ggplot2   4.0.3     v tibble    3.3.1
## v lubridate 1.9.5     v tidyr     1.3.2
## v purrr     1.2.2
## -- Conflicts ------------------------------------------ tidyverse_conflicts() --
## x dplyr::filter() masks stats::filter()
## x dplyr::lag()    masks stats::lag()
## i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

1c) Create a code chunk below this line, and include the line `source("00_requirements.R")`. Print the output of the function `search()` to verify that you successfully installed and included the package in the current environment. We will study more on packages later.

For your project, ensure that you create this `00_requirements.R` file. Every time you need a package, include it in your `00_requirements.R` file. Then, load the packages in each R Markdown file with the `source()` function.

```
source("00_requirements.R")
```

```
## Loading required package: readxl
```

```
## Loading required package: magrittr
```

```
##
## Attaching package: 'magrittr'
```

```
## The following object is masked from 'package:purrr':
##
##     set_names
```

```
## The following object is masked from 'package:tidyr':
##
##     extract
```

```
print(search())
```

```
##  [1] ".GlobalEnv"        "package:magrittr"  "package:readxl"
##  [4] "package:lubridate" "package:forcats"   "package:stringr"
##  [7] "package:dplyr"     "package:purrr"     "package:readr"
## [10] "package:tidyr"     "package:tibble"    "package:ggplot2"
## [13] "package:tidyverse" "package:stats"     "package:graphics"
## [16] "package:grDevices" "package:utils"     "package:datasets"
## [19] "package:methods"   "Autoloads"         "package:base"
```

Step 2: creating external functions file

In your analysis, you may need to create functions that are used throughout your code. For example, you may need to calculate the average of each column in a matrix repeatedly.

2a) In the folder that you created to run DA4, create a plain R-code file and name it `01_matrixFunctions.R`.

2b) Within the file `01_matrixFunctions.R`, paste the code provided in the following chunk, and uncomment the definition of the function `colmean`, which for an `m x n` numeric matrix returns a vector of length `n` that has the average of each column in the matrix. Also define a variable `x_matrix` containing a matrix of dimension `3 x 3` with `1,2,3` in the first column, `4,5,6` in the second, and `7,8,9` on the third.

```
##########################################################
##########################################################
## colmean:
##    For a numeric m x n matrix, calculates the column
##    means.
##
##  INPUTS:
##    A : a numeric matrix of any dimensions m x n.
##
##  OUTPUTS:
##    x : a numeric vector of length n.
##
##########################################################
## NOTE: KEEP THE CODE BELOW COMMENTED IN THIS FILE.
##      UNCOMMENT ONLY IN THE FILE 01_matrixFunctions.R
##########################################################
# colmean <- function(A) {
#  ## Verifying correct inputs.
#  if (!is.numeric(A)) {
#    stop("Error type: must be a numeric matrix.")
#  } else if (!is.matrix(A)) {
#    stop("Error type: object A must be a matrix.")
#  }
#
#  ## Calculation:
#  x <- apply(A, 2, mean)
#  return(x)
#}
```

2c) Create a code chunk below this line, and include the line `source("01_matrixFunctions.R")`. Print the output of the function `ls()` to verify that both `colmean` and the object `x_matrix` are now part of the environment. Finally, print the result of applying `colmean` to the matrix `x_matrix`.

```
source('01_matrixFunctions.R')
exists('colmean')
```

```
## [1] TRUE
```

```
exists('x_matrix')
```

```
## [1] TRUE
```

```r
print(colmean(x_matrix))
```

```
## [1] 2 5 8
```

Step 3: Saving environments

Because data analyses can be complex, it is often best to process data in stages. Often one has to perform cleaning, processing, visualizing, modeling and generating final outcomes of the data analysis separately. Usually, at each step data are modified, and keeping track of how the dataset changes throughout the process is essential.

A good way to keep track of how the data changes at each stage is by saving **R work environments.** This means that you can save the current stage of the R environment, to load later. To practice this, complete the following exercise.

3a) Download the dataset `00_regression_nba.xlsx` to your `DA4` folder. Then, on a chunk of code below, import this dataset into R. (Hint: consider installing and using the package `readxl`)

```r
if (!requireNamespace("readxl", quietly = TRUE)) {
  install.packages('readxl')
}
library(readxl)
nba_data <- read_excel('00_regression_nba-1.xlsx')
```

3b) Create a chunk and rename the variable `"player_height (cm)"` to `height_in` and `"player_weight (kg)"` to `weight_lbs`. Then, convert these columns to the new units. Use the command `rm()` to remove the objects `colmean` and `x_matrix` from the environment.

```r
nba_data$height_in  <- nba_data$`player_height (cm)` / 2.54
nba_data$weight_lbs <- nba_data$`player_weight (kg)` * 2.20462
rm(colmean, x_matrix)
```

3c) Create a chunk and add the command `save.image("12_nbaAnalysisData.RData")`. This will save the environment. Then, add the command `rm(list = ls())`, which empties the environment. Finally, print the outcome of `ls()` to verify that the environment is now empty.

```r
save.image("12_nbaAnalysisData.RData")
rm(list = ls())
print(ls())
```

```
## character(0)
```

3d) Below these instructions, create a code chunk and add the command `load("12_nbaAnalysisData.RData")`. Print the outcome of `ls()` to verify that you restored the variables that used to be in your environment. Then, display the first six rows of the processed dataset, Verify that the names are changed as desired, and they are in the appropriate new units.

```r
load("12_nbaAnalysisData.RData")
print(nba_data)
```

```
## # A tibble: 514 x 5
##    player_name    `player_height (cm)` `player_weight (kg)` height_in weight_lbs
##    <chr>                         <dbl>                <dbl>     <dbl>      <dbl>
##  1 Georges Niang                  201.                104.         79       230.
##  2 George Hill                    190.                 85.3        75       188.
##  3 Gary Trent Jr.                 196.                 94.8        77       209.
##  4 Gary Payton II                 190.                 88.5        75       195.
##  5 Gary Harris                    193.                 95.3        76       210.
##  6 Garrett Temple                 196.                 88.5        77       195.
##  7 Garrison Math~                 196.                 97.5        77       215.
##  8 Giannis Antet~                 211.                110.         83       242.
##  9 Gabe Vincent                   190.                 90.7        75       200.
## 10 Furkan Korkmaz                 201.                 91.6        79       202.
## # i 504 more rows
```

Step 4: Submission and conclusion

Once you finish, you should have a folder that contains the following files, organized according to the file-naming convention described above.

- `00_requirements.R`: contains commands for loading the packages used throughout your analysis. To include in an R Markdown, just use the command `source("00_requirements.R")`.
- `00_regression_nba.xlsx`: data to be analyzed.
- `01_matrixFunctions.R`: contains functions that may be used in one or more R Markdown files. If you need them in a particular file, use the command `source("01_matrixFunctions.R")`.
- `11_nbaAnalysis_dataProcessing.Rmd`: contains the first step of analysis, consisting of transforming the units of the NBA dataset to imperial units.
- `12_nbaAnalysisData.RData`: contains the data processing results of `11_nbaAnalysis_dataProcessing.Rmd`. We can use this data in the next step (for example, visualization in a new file `21_nbaAnalysis_visualization.Rmd`), by loading it via `load("12_nbaAnalysisData.RData")`.

4a) Save all changes, with the structure described before. In the Files pane in the bottom-right corner of RStudio, open the `DA4` folder. Take a screenshot of the folder containing all the aforementioned files. Save this image inside of folder `.../DA4` and include as an image below:

The screenshot must show the files previously created in the `DA4` folder. It must also include the environment pane, with the variables created throughout the exercise. Knit the document and submit.

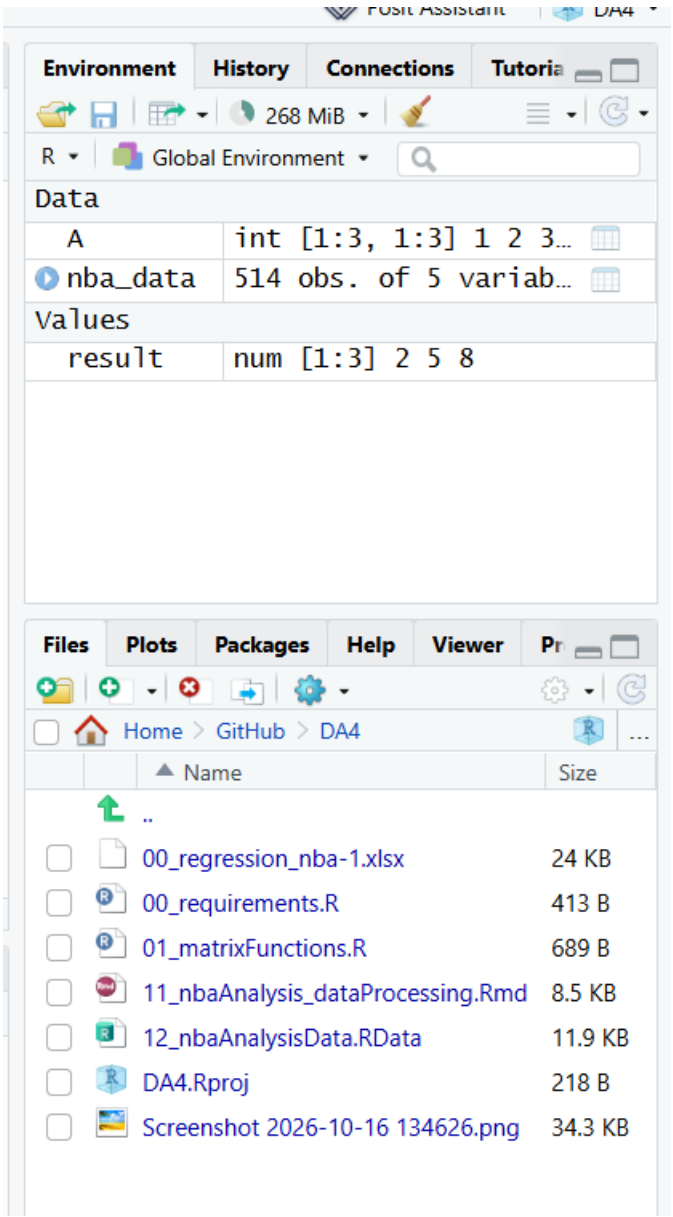


Figure 1: Example image. Substitute with your screenshot here.